Control Flow in R & Python

Lecture 03

Dr. Colin Rundel

Logical & comparison operators

Comparison operators

The syntax is nearly identical. The key difference is that R's comparison operators are *vectorized* (element-wise, returning a logical vector), whereas Python compares two objects and returns a single `bool`.

Comparison	R	Python
less than	`x < y`	`x < y`
greater than	`x > y`	`x > y`
less than or equal to	`x <= y`	`x <= y`
greater than or equal to	`x >= y`	`x >= y`
equal to	`x == y`	`x == y`
not equal to	`x != y`	`x != y`
membership	`x %in% y`	`x in y`

Comparisons

R compares element by element, while Python compares the objects as a whole and uses `in` for membership.

```r
x = c("A", "B", "C")
```

```r
x == "A"
```

```
[1]  TRUE FALSE FALSE
```

```r
"A" %in% x
```

```
[1] TRUE
```

```r
c(1, 2, 3) == c(1, 2, 3)
```

```
[1] TRUE TRUE TRUE
```

```r
c(1, 2, 3) < c(1, 3, 0)
```

```
[1] FALSE  TRUE FALSE
```

```python
x = ["A", "B", "C"]
```

```python
x == "A"
```

```
False
```

```python
"A" in x
```

```
True
```

```python
[1, 2, 3] == [1, 2, 3]
```

```
True
```

```python
[1, 2, 3] < [1, 3, 0]
```

```
True
```

Python's ordered comparisons of lists are *lexicographic* - more on this in a moment, and on recycling and vectorization

Comparing strings

Both languages can order strings, but differently - Python compares by Unicode code point (so ASCII `A`-`Z` sort before ASCII `a`-`z`), while R uses the collation rules of the current locale (roughly alphabetical, with case as a tie breaker).

```r
"A" < "B"
```

```
[1] TRUE
```

```r
"Good" < "Goodbye"
```

```
[1] TRUE
```

```r
"A" < "a"
```

```
[1] FALSE
```

```r
"a" < "B"
```

```
[1] TRUE
```

```r
"Z" < "a"
```

```
[1] FALSE
```

```python
"A" < "B"
```

```
True
```

```python
"Good" < "Goodbye"
```

```
True
```

```python
"A" < "a"
```

```
True
```

```python
"a" < "B"
```

```
False
```

```python
"Z" < "a"
```

```
True
```

See `Sys.getlocale("LC_COLLATE")` and `?Comparison` for R's locale (in the `C` locale R also orders by code point), and `locale.strcoll()` / `locale.strxfrm()` for locale-aware ordering in Python. The same rules apply to `sort()` and

Lexicographic ordering

Ordered comparisons of strings, lists, and tuples in Python, are *lexicographic* (dictionary order) - elements are compared pairwise from the front and the first difference decides the result. If one sequence runs out first, it sorts before the longer one.

```r
"card" < "cart"
```

```
[1] TRUE
```

```r
"cat" < "card"
```

```
[1] FALSE
```

```r
"car" < "card"
```

```
[1] TRUE
```

```python
"card" < "cart"
```

```
True
```

```python
"cat" < "card"
```

```
False
```

```python
"car" < "card"
```

```
True
```

```python
[1, 2, 3] < [1, 3, 0]
```

```
True
```

```python
(1, 2) < (1, 2, 5)
```

```
True
```

Only the elements up to the first difference are ever compared - `[0, "a"] < [1, 2]` is `True`, but `[1, "a"] < [1, 2]`

Logical operators

Operation	R (vectorized)	R (scalar)	Python
and	`x & y`	`x && y`	`x and y`
or	`x | y`	`x || y`	`x or y`
not	`!x`		`not x`
exclusive or	`xor(x, y)`		`x != y`

Python has `&`, `|`, and `~` operators, but they are *bitwise* operators for integers; confusingly, `numpy` and `pandas` overload them as element-wise logical operators (more on this later).

Vectorized vs scalar

In R you choose between the vectorized and scalar forms; in base Python `and` / `or` are always scalar, though with lists they can misleadingly look vectorized.

```r
x = c(TRUE, FALSE, TRUE)
y = c(FALSE, TRUE, TRUE)
```

```r
x | y
```

```
[1] TRUE TRUE TRUE
```

```r
x & y
```

```
[1] FALSE FALSE  TRUE
```

```r
x || y
```

```
Error in `x || y`:
! 'length = 3' in coercion to 'logical(1)'
```

```python
x = [True, False, True]
y = [False, True, True]
```

```python
x or y  # returns x
```

```
[True, False, True]
```

```python
x and y # returns y
```

```
[False, True, True]
```

Since R 4.3, the `&&` and `||` operators throw an error if either side has length greater than 1 (older versions silently used

Short-circuit evaluation

`&&` / `||` in R and `and` / `or` in Python evaluate their left operand first and only evaluate the right operand if the left one does not already decide the outcome.

```r
FALSE && stop("Error")
```

```
[1] FALSE
```

```r
TRUE || stop("Error")
```

```
[1] TRUE
```

```r
TRUE && stop("Error")
```

```
Error:
! Error
```

```r
FALSE || stop("Error")
```

```
Error:
! Error
```

```python
False and 1/0
```

```
False
```

```python
True or 1/0
```

```
True
```

```python
True and 1/0
```

```
ZeroDivisionError: division by zero
```

```python
False or 1/0
```

```
ZeroDivisionError: division by zero
```

Guarding with short-circuits

This makes them useful as *guards*, where an earlier condition rules out inputs that would cause a later condition to error.

```r
x = "abc"
abs(x) > 1
```

```
Error in `abs()`:
! non-numeric argument to mathematical function
```

```r
is.numeric(x) && abs(x) > 1
```

```
[1] FALSE
```

```r
x = NULL
abs(x) > 1
```

```
Error in `abs()`:
! non-numeric argument to mathematical function
```

```r
!is.null(x) && abs(x) > 1
```

```
[1] FALSE
```

```python
x = "abc"
abs(x) > 1
```

```
TypeError: bad operand type for abs(): 'str'
```

```python
isinstance(x, (int, float)) and abs(x) > 1
```

```
False
```

```python
x = None
abs(x) > 1
```

```
TypeError: bad operand type for abs(): 'NoneType
```

```python
x is not None and abs(x) > 1
```

```
False
```

Truthiness and short-circuiting

Python's `and` and `or` accept any values, not just `bool`s. They check the *truthiness* of `x` and short-circuiting decides which value comes back:

- `x and y` - if `x` is falsy the result is `x`, otherwise it is `y`
- `x or y` - if `x` is truthy the result is `x`, otherwise it is `y`

Either way the result is one of the original values, unchanged, whereas R's `&&` and `||` always produce a single `TRUE`, `FALSE`, or `NA`.

```py
1 and 2    # 1 is truthy -> 2
```

```
2
```

```py
0 and 2    # 0 is falsy  -> 0
```

```
0
```

```py
[] and [1] # [] is falsy -> []
```

```
[]
```

```py
0 or "default"  # 0 is falsy  -> "default
```

```
'default'
```

```py
"" or "default" # "" is falsy -> "default
```

```
'default'
```

```py
"abc" or "default" # "abc" is truthy -> "ab
```

```
'abc'
```

Fallback values

```py
x or default
```

is a common Python idiom for supplying a default when `x` is falsy. R 4.4 added the null coalescing operator `%||%` for the narrower case where `x` is `NULL`.

```py
name = ""
name or "anonymous"
```

```
'anonymous'
```

```R
name = NULL
name %||% "anonymous"
```

```
[1] "anonymous"
```

```py
name = "Colin"
name or "anonymous"
```

```
'Colin'
```

```R
name = "Colin"
name %||% "anonymous"
```

```
[1] "Colin"
```

Conditionals

`if` and `else`

```r
x = 3
if (x > 0) {
  print("x is positive")
}
```

```
[1] "x is positive"
```

```r
if (x > 0) {
  print("x is positive")
} else {
  print("x is not positive")
}
```

```
[1] "x is positive"
```

```python
x = 3
if x > 0:
    print("x is positive")
```

```
x is positive
```

```python
if x > 0:
    print("x is positive")
else:
    print("x is not positive")
```

```
x is positive
```

R wraps the condition in `()` and the body in `{}`, while Python ends the condition with `:` and the body is the following *indented* block.

In R the `{}` are optional for a single-expression body, e.g. `if (x > 0) print("x is positive")`, but the we and the

`else if` and `elif`

Conditions are checked in order and only the *first* true branch runs; the optional `else` branch runs if no other branch triggered.

R

```r
x = 0
if (x < 0) {
  print("x is negative")
} else if (x > 0) {
  print("x is positive")
} else {
  print("x is zero")
}
```

```
[1] "x is zero"
```

py

```python
x = 0
if x < 0:
    print("x is negative")
elif x > 0:
    print("x is positive")
else:
    print("x is zero")
```

```
x is zero
```

Conditionals as expressions

R's `if` is an expression that returns the value of the evaluated branch, so it can be used directly in an assignment. Python's `if` is a *statement* that returns nothing; instead there is a separate *conditional expression*, `a if cond else b`.

R

```r
x = 5
s = if (x %% 2 == 0) x / 2 else 3 * x + 1
s
```

```
[1] 16
```

py

```python
x = 5
s = x / 2 if x % 2 == 0 else 3 * x + 1
s
```

```
16
```

Both are equivalent to assigning within each branch,

R

```r
if (x %% 2 == 0) {
  s = x / 2
} else {
  s = 3 * x + 1
}
s
```

```
[1] 16
```

py

```python
if x % 2 == 0:
    s = x / 2
else:
    s = 3 * x + 1

s
```

```
16
```

Conditionals are *not* vectorized

```r
x = c(1, 3)
```

```r
if (x == 1) print("x is 1!")
```

```
Error in `if (x == 1) ...`:
! the condition has length > 1
```

```r
if (x == 3) print("x is 3!")
```

```
Error in `if (x == 3) ...`:
! the condition has length > 1
```

```python
x = [1, 3]
```

```python
if x == 1:
    print("x is 1!")
else:
    print("x is not 1!")
```

```
x is not 1!
```

```python
if [False, False]:
    print("non-empty lists are truthy")
```

```
non-empty lists are truthy
```

Since R 4.2, `if` throws an error if the condition has length > 1 (older versions used the first value with a warning).

Python does not automatically apply the comparison element-wise - `x == 1` is simply `False`, and any non-empty list is truthy regardless of its contents.

Collapsing logical vectors

Both languages provide `any()` and `all()` for reducing multiple logical values to a single one.

```r
x = c(3, 4, 1)
x >= 2
```

```
[1]  TRUE  TRUE FALSE
```

```r
any(x >= 2)
```

```
[1] TRUE
```

```r
all(x >= 2)
```

```
[1] FALSE
```

```r
if (any(x == 3)) print("x contains 3!")
```

```
[1] "x contains 3!"
```

```python
x = [3, 4, 1]
any([True, True, False])
```

```
True
```

```python
all([True, True, False])
```

```
False
```

```python
if 3 in x: print("x contains 3!")
```

```
x contains 3!
```

Conditionals and truthiness

Python's conditionals accept any object and use its *truthiness*. R's `if` requires a single logical value, though it will coerce other values that `as.logical()` recognizes.

```r
if (1) "yes" else "no"
```

```
[1] "yes"
```

```r
if (0) "yes" else "no"
```

```
[1] "no"
```

```r
if ("TRUE") "yes" else "no"
```

```
[1] "yes"
```

```r
if ("abc") "yes" else "no"
```

```
Error in `if ("abc") ...`:
! argument is not interpretable as logical
```

```r
if (NULL) "yes" else "no"
```

```
Error in `if (NULL) ...`:
! argument is of length zero
```

```python
"yes" if 1 else "no"
```

```
'yes'
```

```python
"yes" if 0 else "no"
```

```
'no'
```

```python
"yes" if "abc" else "no"
```

```
'yes'
```

```python
"yes" if [] else "no"
```

```
'no'
```

```python
"yes" if None else "no"
```

```
'no'
```

Vectorized conditionals

R's `ifelse()` is the element-wise counterpart to `if` - given a logical vector it returns a vector of the same length built from the `yes` and `no` arguments.

```r
x = c(-2, 0, 3)
ifelse(x > 0, "positive", "non-positive")
```

```
[1] "non-positive" "non-positive" "positive"
```

```r
ifelse(x > 0, x, -x)
```

```
[1] 2 0 3
```

Base Python has no equivalent; the idiomatic approach is a list comprehension with a conditional expression (next time).

Conditionals and missing values

- R - `NA` is sticky in comparisons and `if` errors on `NA` rather than guessing. `any()` and `all()` follow the `|` and `&` rules.
- Python - `None` supports equality tests, but ordering it with a number raises `TypeError`. For `nan`, ordered comparisons and `==` are `False`; `!=` is `True`; and despite comparing equal to nothing, `nan` is truthy.

Check for missing values directly with `is.na()`, `x is None`, or `math.isnan()`.

R

```r
x = NA
x > 1
```

```
[1] NA
```

R

```r
if (x > 1) print("x is big")
```

```
Error in `if (x > 1) ...`:
! missing value where TRUE/FALSE needed
```

R

```r
any(c(1, NA, 4) >= 3)
```

```
[1] TRUE
```

R

```r
all(c(1, NA, 4) >= 1)
```

```
[1] NA
```

py

```python
x = None
```

py

```python
if x > 1: print("x is big")
```

```
TypeError: '>' not supported between instances of 'None
```

py

```python
if x is None: print("x is missing")
```

```
x is missing
```

py

```python
(math.nan > 1, math.nan == math.nan, math.nan !=
```

```
(False, False, True)
```

py

```python
bool(math.nan)
```

```
True
```

Multi-way branching

R's `switch()` selects a branch based on a character (or integer) value, while Python 3.10+ has the `match` statement (which also supports much more general *structural pattern matching*).

R

```r
x = "b"
switch(x,
  a = "apple",
  b = "banana",
  "unknown"
)
```

```
[1] "banana"
```

py

```python
x = "b"
match x:
    case "a":
        print("apple")
    case "b":
        print("banana")
    case _:
        print("unknown")
```

```
banana
```

`switch()` uses an unnamed final argument as the default (without one an unmatched value returns `NULL`, invisibly), `case`

Errors

Signaling conditions

R has several ways of communicating with the user beyond `print()` / `cat()`, each of which is a *condition* that can be handled programmatically:

- `message()` - diagnostic messages (sent to stderr)
- `warning()` - something unexpected but not fatal, execution continues
- `stop()` - an error, execution halts

Python has rough equivalents in `print()` (or the `logging` module), `warnings.warn()`, and `raise` with an *exception* object.

```r
message("Starting")
```

```
Starting
```

```r
warning("Something unexpected")
```

```
Warning: Something unexpected
```

```python
import warnings
print("Starting")
```

```
Starting
```

```python
warnings.warn("Something unexpected")
```

```
<string>:1: UserWarning: Something unexpected
```

Errors stop a Quarto document from rendering unless the chunk sets `#| error: true` (which is how these slides show

Raising errors

```r
x = -1
if (x < 0) {
  stop("x must be non-negative")
}
```

```
Error:
! x must be non-negative
```

```python
x = -1
if x < 0:
    raise ValueError("x must be non-ne
```

```
ValueError: x must be non-negative
```

Both languages also have a shorthand for checking assumptions - R's `stopifnot()` and Python's `assert` statement:

```r
stopifnot(x >= 0)
```

```
Error:
! x >= 0 is not TRUE
```

```r
stopifnot("x must be non-negative" =
```

```
Error:
! x must be non-negative
```

```python
assert x >= 0
```

```
AssertionError
```

```python
assert x >= 0, "x must be non-negativ
```

```
AssertionError: x must be non-negative
```

Python's `assert` statements are stripped when running with `python -O`, so use them for development-time checks

Errors vs exceptions

Python exceptions are objects with a class hierarchy - the class describes what went wrong and allows handling to be selective. R errors are, by default, all of the same class (`simpleError`) and are distinguished only by their message.

```r
"abc" + 1
```

```
Error in `"abc" + 1`:
! non-numeric argument to binary operator
```

```r
log("abc")
```

```
Error in `log()`:
! non-numeric argument to mathematical function
```

```r
sum("abc")
```

```
Error in `sum()`:
! invalid 'type' (character) of argument
```

```r
undefined_var
```

```
Error:
! object 'undefined_var' not found
```

```python
"abc" + 1
```

```
TypeError: can only concatenate str (not "int") to str
```

```python
int("abc")
```

```
ValueError: invalid literal for int() with base 10: 'ab
```

```python
[1, 2, 3][5]
```

```
IndexError: list index out of range
```

```python
undefined_var
```

```
NameError: name 'undefined_var' is not defined
```

See the Python docs for the full exception hierarchy. Classed conditions are possible in R (e.g. via `rlang::abort()` /

Handling errors

R's `try()` evaluates an expression and, instead of halting, returns a `try-error` object if an error occurs. Python's `try` / `except` block runs the `except` code only if a matching exception was raised in the `try` body.

```r
x = try(log("a"), silent = TRUE)
class(x)
```

```
[1] "try-error"
```

```r
cat(x)
```

```
Error in log("a") : non-numeric argument t
```

```r
inherits(x, "try-error")
```

```
[1] TRUE
```

```python
try:
    x = math.log("a")
except TypeError as e:
    print("Caught:", e)
    x = math.nan
```

```
Caught: must be real number, not str
```

```python
x
```

```
nan
```

Exercise 1

Without running the code, what do you expect the output (or error) to be for each of the listed values of `x`?

R

```r
if (x > 10 || x < -10) {
  stop("Input too big")
} else if (x %in% c(2, 3, 5, 7)) {
  cat("Input is prime!\n")
} else if (x %% 2 == 0) {
  cat("Input is even!\n")
} else if (x %% 2 == 1) {
  cat("Input is odd!\n")
}
```

R

```r
x = 1
x = 3
x = -1
x = -3
x = 1:2
x = "0"
x = "zero"
```

py

```python
if x > 10 or x < -10:
    raise ValueError("Input too big")
elif x in [2, 3, 5, 7]:
    print("Input is prime!")
elif x % 2 == 0:
    print("Input is even!")
elif x % 2 == 1:
    print("Input is odd!")
```

py

```python
x = 1
x = 3
x = -1
x = -3
x = [1, 2]
x = "0"
x = 2.5
```

Loops

`for` loops

R's `for` iterates over the elements of a vector (or list), while Python's `for` iterates over the elements of any *iterable* object (lists, tuples, strings, ranges, dictionaries, files, …).

R

```r
for (w in c("Hello", "world!")) {
  cat(w, ":", nchar(w), "\n")
}
```

```
Hello : 5
world! : 6
```

R

```r
total = 0
for (v in c(1, 2, 3, 4)) {
  total = total + v
}
total
```

```
[1] 10
```

py

```python
for w in ["Hello", "world!"]:
    print(w, ":", len(w))
```

```
Hello : 5
world! : 6
```

py

```python
total = 0
for v in (1, 2, 3, 4):
    total += v
total
```

```
10
```

Integer sequences

Loops over indices need a sequence of integers - R has `:`, `seq()`, `seq_len()`, and `seq_along()`, while Python has `range()`.

R

```r
1:5
```

```
[1] 1 2 3 4 5
```

R

```r
seq(1, 10, by = 3)
```

```
[1]  1  4  7 10
```

R

```r
seq_len(3)
```

```
[1] 1 2 3
```

R

```r
seq_along(c("a", "b", "c"))
```

```
[1] 1 2 3
```

py

```python
range(5)
```

```
range(0, 5)
```

py

```python
list(range(5))
```

```
[0, 1, 2, 3, 4]
```

py

```python
list(range(1, 11, 3))
```

```
[1, 4, 7, 10]
```

py

```python
list(range(5, 0, -1))
```

```
[5, 4, 3, 2, 1]
```

Looping over indices

The R idiom is `seq_along(x)`. Python's equivalent is `range(len(x))`, but `enumerate()` is preferred as it yields the index *and* the value together (as a tuple, which is unpacked into `i` and `v` below).

R

```r
x = c("a", "b", "c")
for (i in seq_along(x)) {
  cat(i, x[i], "\n")
}
```

```
1 a
2 b
3 c
```

py

```python
x = ["a", "b", "c"]
for i in range(len(x)):
    print(i, x[i])
```

```
0 a
1 b
2 c
```

py

```python
for i, v in enumerate(x):
    print(i, v)
```

```
0 a
1 b
2 c
```

Avoid `1:length(x)`

The common R idiom `1:length(x)` fails for empty vectors since `1:0` is `c(1, 0)` - use `seq_along()` or `seq_len()` instead. Python's `range(len(x))` is safe since `range(0)` is empty.

```r
x = integer()
1:length(x)
```

```
[1] 1 0
```

```r
seq_along(x)
```

```
integer(0)
```

```r
for (i in 1:length(x)) print(i)
```

```
[1] 1
[1] 0
```

```r
for (i in seq_along(x)) print(i)
```

```py
x = []
list(range(len(x)))
```

```
[]
```

```py
for i in range(len(x)): print(i)
```

Multiple sequences

Python's `zip()` iterates over multiple sequences together, stopping at the shortest. R has no direct equivalent - index with `seq_along()` instead (though vectorization usually makes this unnecessary).

R

```r
x = c(1, 2, 3)
y = c("a", "b", "c")
for (i in seq_along(x)) {
  cat(x[i], y[i], "\n")
}
```

```
1 a
2 b
3 c
```

py

```python
x = [1, 2, 3]
y = ["a", "b", "c"]
for a, b in zip(x, y):
    print(a, b)
```

```
1 a
2 b
3 c
```

py

```python
list(zip([1, 2, 3, 4], "ab"))
```

```
[(1, 'a'), (2, 'b')]
```

`zip()` returns a lazy iterator (hence the `list()` to see its contents). See `itertools.zip_longest()` to iterate until the

`while` loops

Repeat the body as long as the condition is `TRUE` / truthy - the condition is checked before each iteration, so the body may never run.

```r
i = 1
while (i < 100) {
  i = i * 2
}
i
```

```
[1] 128
```

```python
i = 1
while i < 100:
    i *= 2
i
```

```
128
```

R also has `repeat`, which loops forever until a `break` - the Python idiom for this is `while True:`.

```r
i = 1
repeat {
  i = i * 2
  if (i >= 100) break
}
i
```

```
[1] 128
```

```python
i = 1
while True:
    i *= 2
    if i >= 100:
        break
i
```

```
128
```

`break` and `next` / `continue`

`break` exits the (innermost) loop entirely, while `next` in R and `continue` in Python skip the rest of the current iteration.

R

```r
for (i in 1:10) {
  if (i %% 3 == 0) next
  cat(i, "")
}
```

```
1 2 4 5 7 8 10
```

py

```python
for i in range(1, 11):
    if i % 3 == 0:
        continue
    print(i, end=" ")
```

```
1 2 4 5 7 8 10
```

R

```r
for (i in 1:10) {
  if (i %% 3 == 0) break
  cat(i, "")
}
```

```
1 2
```

py

```python
for i in range(1, 11):
    if i % 3 == 0:
        break
    print(i, end=" ")
```

```
1 2
```

Loop `else` and `pass`

Two Python-only constructs - loops can have an `else` clause that runs when the loop finishes *without* a `break`, and `pass` is a no-op placeholder for where a statement is syntactically required.

```py
for n in range(2, 10):
    for x in range(2, n):
        if n % x == 0:
            print(n, "=", x, "*", n //
            break
    else:
        print(n, "is prime")
```

```py
x = -3
if x < 0:
    pass
elif x % 2 == 0:
    print("x is even")
else:
    print("x is odd")
```

```
2 is prime
3 is prime
4 = 2 * 2
5 is prime
6 = 2 * 3
7 is prime
8 = 2 * 4
9 = 3 * 3
```

Loop `else` example from the Python Tutorial. R has no loop `else`, and an empty block `{}` (or `NULL`) serves as a

Building up results

```r
res = c()
for (i in 1:5) {
  res = c(res, i^2)
}
res
```

```
[1]  1  4  9 16 25
```

```python
res = []
for i in range(1, 6):
    res.append(i**2)
res
```

```
[1, 4, 9, 16, 25]
```

Growing an R vector with `c()` copies it every iteration (quadratic runtime), so for longer loops preallocate (`numeric(n)`, `character(n)`, `vector("list", n)`) and assign by index. Python's `list.append()` is amortized constant time, so appending is idiomatic.

```r
res = numeric(5)
for (i in seq_along(res)) {
  res[i] = i^2
}
res
```

```
[1]  1  4  9 16 25
```

In practice most loops like these are replaced by vectorized operations, functional tools (`lapply()`, `purrr::map()`), or

Exercise 2

To the right are vectors containing all prime numbers between 2 and 100 and some values `x` we would like to check for primality.

Using *nested* loops, write code in both R and Python that prints only the values of `x` that are *not* prime - without using subsetting, `%in%`, or `in`.

In Python, try using the loop `else` clause; in R you will need a flag variable.

R

```r
primes = c( 2,  3,  5,  7, 11, 13, 17, 19, 23,
           29, 31, 37, 41, 43, 47, 53, 59, 61,
           67, 71, 73, 79, 83, 89, 97)
x = c(3, 4, 12, 19, 23, 51, 61, 63, 78)
```

py

```python
primes = [ 2,  3,  5,  7, 11, 13, 17, 19, 23,
          29, 31, 37, 41, 43, 47, 53, 59, 61,
          67, 71, 73, 79, 83, 89, 97]
x = [3, 4, 12, 19, 23, 51, 61, 63, 78]
```

Comparing R & Python

Control flow summary

Construct	R	Python
conditional	`if` / `else if` / `else`	`if` / `elif` / `else`
conditional expression	`if (cond) a else b`	`a if cond else b`
vectorized conditional	`ifelse()`	comprehension, `np.where()`
multi-way branch	`switch()`	`match` / `case`
for loop	`for (x in vec) {}`	`for x in iterable:`
while loop	`while (cond) {}`	`while cond:`
infinite loop	`repeat {}`	`while True:`
skip iteration	`next`	`continue`
exit loop	`break`	`break`
integer sequences	`:`, `seq_len()`, `seq_along()`	`range()`
index + value	`seq_along()` + `x[i]`	`enumerate()`
multiple sequences	`seq_along()` + indexing	`zip()`
reduce logicals	`any()`, `all()`	`any()`, `all()`
blocks	`{ }`	`:` + indentation

Error handling summary

Concept	R	Python
message	`message()`	`print()`, `logging`
warning	`warning()`	`warnings.warn()`
error	`stop()`	`raise SomeError()`
assertion	`stopifnot()`	`assert`
catch	`try()`, `tryCatch()`	`try` / `except`
always run	`tryCatch(finally = )`, `on.exit()`	`finally`
error object	condition (`simpleError`)	exception (subclass of `Exception`)
error message	`conditionMessage(e)`	`str(e)`

Takeaways

- R's comparison and logical operators (`==`, `&`, `|`) are vectorized; `&&` / `||` in R and `and` / `or` in Python are scalar and short-circuit.
- `if` in R needs exactly one `TRUE` / `FALSE` - use `any()`, `all()`, or `ifelse()` for vectors. Python's `if` tests the *truthiness* of any object.
- Loop syntax is nearly identical - the differences are blocks (`{}` vs indentation), `seq_along()` vs `range()` / `enumerate()`, and `next` vs `continue`.
- Errors are *conditions* in R (`stop()`, `tryCatch()`) and typed *exception* objects in Python (`raise`, `try` / `except`).